Kuwait University

Computer Engineering Department

CpE-433 Computer Vision

L5/L6 Fundamentals Practice

PRACTICE – NOT GRADED

Conventions (unless stated otherwise):
kernels are applied by **cross-correlation**, as `cv2.filter2D` does;
x points right and y points **down**; and the Sobel kernels are

$$S_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}, \qquad S_y = S_x^\top = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}.$$

1. (6 points) **Filtering Basics (L5)**

   (a) (1 point) In one or two sentences, what does a smoothing (blurring) filter do to an image? Give two reasons why we smooth an image.

   (b) (2 points) Apply the 3×3 **mean** filter $\frac{1}{9}\mathbf{1}_{3\times3}$ to the patch below. What is the output at the center pixel?

$$P = \begin{bmatrix} 2 & 4 & 6 \\ 8 & 10 & 12 \\ 14 & 16 & 18 \end{bmatrix}$$

   (c) (1 point) Write the 3×3 delta (impulse) kernel δ. What is $f * \delta$?

   (d) (1 point) ___ The entries of the 3×3 mean filter add up to 1. Justify, and explain why this matters.

(e) (1 point) ___ A Gaussian filter with a larger σ blurs the image more. Justify.

spreads weights over a wider neighborhood

2. (6 points) **Noise and the Median Filter (L5)**

(a) (1 point) Name the two types of noise discussed in L5 and describe each in one sentence.

(b) (2 points) The 3×3 neighborhood of a pixel is shown below. Compute the output at the center pixel of (i) a 3×3 **median** filter and (ii) a 3×3 **mean** filter.

$$P = \begin{bmatrix} 5 & 6 & 5 \\ 7 & 0 & 6 \\ 5 & 7 & 6 \end{bmatrix}$$

(c) (1 point) The center pixel in part (b) is a "pepper" pixel. Which filter handled it better, and why?

median, b it removes outliers while keeping edge better

(d) (1 point) ___ The median filter is a linear filter. Justify.

No, nonlinear, doesn't satisfy linearity prop in general

(e) (1 point) An image is blurred with a Gaussian of $\sigma = 5$ and then again with $\sigma = 12$. What single σ gives the same result?

3. (4 points) **Sharpening (L5)**

(a) (1 point) Write the unsharp masking formula. In words, what is the "detail" term?

$f + (f * G)$

$\div 3 = \square$ $\quad 40 - \square = \triangle$

$40 + \alpha(\triangle)$

(b) (2 points) A 1-D signal is $f = [10, 10, 40, 10, 10]$. Using a 3-sample mean as the blur and $a = 1$, compute the sharpened value at the center sample (value 40).

1) Blur

$\frac{10+40+10}{3} = 20$

2) details

$40 - 20 = 20$

Sharpen

$40 + \alpha(F - G)$

$40 + (1)(20) = 60$

(c) (1 point) ___ Sharpening also makes noise more visible. Justify.

4. (8 points) **Image Gradients (L6)**

(a) (1 point) What is an edge in an image? Why does taking a derivative help us find edges?

rapid change

(b) (1 point) A 1-D signal is $f = [10,\ 10,\ 10,\ 50,\ 50,\ 50]$ (indices 0 to 5). Compute the finite difference $f(x+1) - f(x)$ for $x = 0, \ldots, 4$. Where is the edge?

(c) (2 points) Apply the Sobel kernels S_x and S_y (see the cover page) to the patch below. Compute g_x and g_y at the center pixel.

$$P = \begin{bmatrix} 0 & 0 & 100 \\ 0 & 0 & 100 \\ 0 & 0 & 100 \end{bmatrix}$$



-1 0 1
-2 0 2
-1 0 1

(d) (2 points) Compute the gradient magnitude $\|\nabla f\| = \sqrt{g_x^2 + g_y^2}$ and direction $\theta = \text{atan2}(g_y, g_x)$ for: (i) $g_x = 0$, $g_y = 7$; (ii) $g_x = 3$, $g_y = 3$.

(e) (1 point) Which Sobel output, g_x or g_y, responds strongly to **vertical** edges? Explain.

(f) (1 point) ___ We smooth an image before computing its gradient because derivatives amplify noise. Justify.

5. (6 points) **The Canny Edge Detector (L6)**

(a) (2 points) Put these Canny stages in the correct order: *hysteresis*, *non-maximum suppression*, *Gaussian smoothing*, *double threshold*, *gradient magnitude and direction*.

(b) (1 point) In one sentence, what does non-maximum suppression do, and what does the result look like?

(c) (2 points) With a high threshold $T = 80$ and a low threshold $t = 30$, label each gradient magnitude as *strong*, *weak*, or *discarded*: 95, 50, 20, 30, 80.

(d) (1 point) One row of pixels after non-maximum suppression has the magnitudes

$$[\,90,\ 50,\ 0,\ 45\,].$$

Using $T = 80$, $t = 30$, and hysteresis, which pixels are edges?

Appendix: Background and References

Everything you need to solve the practice problems, in one place.

A. Kernels and Filtering (L4–L5)

A.1 Cross-correlation vs. convolution. To filter image f with kernel H, place the kernel over each pixel's neighborhood, **multiply** matching entries, and **add** the products:

$$\text{cross-correlation (}\texttt{cv2.filter2D}\text{):}\quad g(i,j) = \sum_{u=-1}^{1}\sum_{v=-1}^{1} H(u,v)\, f(i+u,\, j+v)$$

$$\text{convolution (kernel flipped by 180°):}\quad g(i,j) = \sum_{u=-1}^{1}\sum_{v=-1}^{1} H(u,v)\, f(i-u,\, j-v)$$

For *symmetric* kernels (mean, Gaussian, δ) the two are identical. For *antisymmetric* kernels (Sobel), flipping negates the kernel, so the output changes sign.

A.2 Linearity. For any kernels A, B and constant c:

$$f * A + f * B = f * (A + B), \qquad c\,(f * A) = f * (c\,A), \qquad (f * A) * B = f * (A * B).$$

These rules let several filtering steps be combined into a single kernel. Unsharp masking (Section C) uses them.

A.3 The delta (impulse) kernel δ. δ is all zeros except for a single 1 at the center:

$$\delta = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}, \qquad f * \delta = f.$$

It is the "do nothing" kernel: every neighbor is multiplied by 0 and the center by 1, so each output pixel equals the input pixel. It plays the role in convolution that the number 1 plays in multiplication.

- Adding $c\,\delta$ to a kernel just adds c to its **center** entry. That's how $K = (1+a)\,\delta - a\,G$ in Section C is built.
- δ is **not** the identity matrix. `np.eye(3)` has 1s along the whole diagonal and, used as a kernel, adds up three diagonal pixels. In NumPy: `d = np.zeros((3, 3)); d[1, 1] = 1`.

A.4 What a kernel's sum tells you. On a flat region where every pixel equals c, the output is $c \times$ (sum of kernel entries).

Kernel sum	Effect on a flat region	Examples
1	unchanged (brightness preserved)	mean, Gaussian, δ, unsharp kernel K
0	becomes 0 (no response)	Sobel S_x, S_y, Laplacian

B. Smoothing, Denoising, and Sampling (L5)

B.1 Mean (box) filter. $G = \frac{1}{9}\begin{bmatrix}1 & 1 & 1\\ 1 & 1 & 1\\ 1 & 1 & 1\end{bmatrix}$ replaces each pixel with the average of its 3×3 neighborhood.

B.2 Gaussian filter. $G_\sigma(x, y) = \dfrac{1}{2\pi\sigma^2}\, e^{-(x^2+y^2)/(2\sigma^2)}$. Nearby pixels get more weight than distant ones, and a larger σ means more blur. Rule of thumb: the kernel should extend about 3σ on each side of the center.

- **Cascading:** blurring with σ_1 and then σ_2 equals one blur with $\sigma = \sqrt{\sigma_1^2 + \sigma_2^2}$. The *variances* add, not the σ's.
- **Choosing σ:** too small leaves noise, and too large destroys detail. The best σ grows with the noise level.

B.3 Median filter. **Sort** the window and output the **middle** value. The median is *non-linear* ($\text{median}(f+g) \neq \text{median}(f) + \text{median}(g)$ in general), so no kernel can implement it.

B.4 Which filter for which noise?

Noise type	What it looks like	Best filter
Salt-and-pepper (impulse)	isolated pixels at 0 or 255	**median**: removes outliers, keeps edges sharp
Gaussian	small random changes at every pixel	**Gaussian / mean**: averages the noise out

B.5 Aliasing: blur before downsampling. Keeping every other pixel cannot represent detail finer than 2 pixels. That detail turns into false coarse patterns (aliasing, moiré). Always blur (e.g., Gaussian) *before* subsampling, which is what image pyramids do.

C. Sharpening: Unsharp Masking (L5)

$$\text{sharp} = f + a\,\underbrace{(f - f * G)}_{\text{detail}} = f * K, \qquad K = (1+a)\,\delta - a\,G.$$

- The *detail* is what the blur removed: edges, texture, and noise. a sets how much of it is added back.
- K always sums to $(1 + a) - a = 1$, so flat regions are unchanged.
- **Halos:** near an edge the detail overshoots on both sides, giving a too-bright band and a too-dark band. Their size is proportional to a. A larger a also amplifies noise.

D. Image Gradients (L6)

D.1 Image coordinates. Pixel (r, c) is row r, column c, numbered from 0 at the top-left. x increases to the **right** (columns) and y increases **downward** (rows), as in NumPy and OpenCV.

D.2 Sobel kernels and a shortcut.

$$S_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix} \qquad S_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

- g_x = (right column − left column), with weights $1, 2, 1$ from top to bottom. It responds to **vertical** edges.
- g_y = (bottom row − top row), with weights $1, 2, 1$ from left to right. It responds to **horizontal** edges.

D.3 Magnitude and direction.

$$\|\nabla f\| = \sqrt{g_x^2 + g_y^2}, \qquad \theta = \text{atan2}(g_y,\, g_x).$$

Use atan2, which checks both signs to choose the correct quadrant, not $\arctan(g_y/g_x)$. The gradient points from **dark to bright**, and the edge runs **perpendicular** to it. Since y points down:

θ	gradient points	edge runs
0°	right	vertically
45°	down-right	bottom-left to top-right
90°	down	horizontally
135°	down-left	top-left to bottom-right

θ and $\theta + 180°$ describe the same edge, so Canny folds θ into $[0°, 180°)$.

E. The Canny Edge Detector (L6)

E.1 Pipeline. **Smooth** (Gaussian) → **differentiate** (Sobel: magnitude and θ) → **thin** (non-maximum suppression) → **classify** (double threshold) → **link** (hysteresis).

E.2 Non-maximum suppression (NMS). Round θ to the nearest of $0°, 45°, 90°, 135°$. Keep the pixel only if its magnitude is $\geq$ both neighbors **along the gradient**, that is, across the edge:

rounded θ	compare with (y pointing down)
0°	left and right
45°	up-left and down-right
90°	up and down
135°	up-right and down-left

E.3 Double threshold and hysteresis. With a high threshold T and a low threshold t:

- $m \geq T$: **strong**, always an edge.
- $t \leq m < T$: **weak**, kept only if connected to a strong pixel through a chain of weak or strong pixels.

- $m < t$: **discarded**, never an edge. Hysteresis cannot rescue it.

8-connectivity: a pixel's neighbors are the 8 pixels around it, diagonals included. Whether a weak pixel survives depends on *connection*, not on its value.

F. References

[1] CpE-433 lecture slides: L5 *Smoothing, Denoising, and Sharpening* and L6 *Image Gradients and Edge Detection*, Fall 2026.

[2] CpE-433 Week 2 Colab notebook: *Filtering and Edges*.

[3] R. Szeliski, *Computer Vision: Algorithms and Applications*, 2nd ed., Springer, 2022. Free online at `szeliski.org/Book`. Relevant sections: §3.2 Linear filtering; §3.3.1 Non-linear filtering (median); §3.5.2 Decimation (blur before downsampling); §7.2.1 Edge detection.

[4] J. Canny, "A Computational Approach to Edge Detection," *IEEE Trans. Pattern Analysis and Machine Intelligence*, vol. PAMI-8, no. 6, pp. 679–698, 1986.

[5] OpenCV documentation (`docs.opencv.org`): `cv2.filter2D`, `cv2.GaussianBlur`, `cv2.medianBlur`, `cv2.Sobel`, `cv2.Canny`.

[6] S. Nayar, *First Principles of Computer Vision* (Columbia University, YouTube): short videos on image filtering and edge detection.